

ICT162

Proctored Online Exam - January Semester 2026

Object Oriented Programming

Thursday, 23 April 2026

04:00 pm - 06:00 pm

Time allowed: 2 hours

INSTRUCTIONS TO STUDENTS:

1. This examination contains **13** questions.
2. Answer all parts of each question in the corresponding answer box.
3. All answers must be typed in the answer box and submitted in Examena.
4. This is **a/an Closed-book** examination.
5. To prevent plagiarism and collusion, your submission will be reviewed by Turnitin. The Turnitin report will only be made available to the marker and you will not be able to view it.
6. The University takes plagiarism and collusion seriously, and your Turnitin report will be examined thoroughly as part of the marking process.
7. One (1) A4-sized (double-sided) hardcopy Cheat Sheet allowed.

At the end of the examination.

To end the examination, please click on the "Submit and end exam" button to submit your answers.

**THE UNIVERSITY RESERVES THE RIGHT NOT TO MARK YOUR
SCRIPT IF YOU FAIL TO FOLLOW THESE INSTRUCTIONS.**

(Full marks: 100)

Question 1

Apply the principles of object-oriented programming principles in answering question 1-10.

```
1 class Gadget:
2     _os = "PyOS"
3     def __init__(self):
4         self._os = "uOS"
5
6 g1 = Gadget()
7 g2 = Gadget()
8 del g2._os
9
10 print(Gadget._os, g1._os, g2._os)
```

What is printed?

- A. PyOS uOS uOS
- B. PyOS uOS PyOS
- C. uOS uOS PyOS
- D. AttributeError at line 8

(2 marks)

Question 2

```
class A:
    def __init__(self, v=0):
        self._v = v
class B(A):
    def __init__(self, x):
        self._v = 0 # line 1
        self._x = x

b = B(5)
print(b._v, b._x)
```

Which replacement for line 1 keeps the code correct and produces output “0 5”?

- A. A.__init__(self)
- B. super().__init__()
- C. super().__init__(x)
- D. A.__init__(x)

(2 marks)

Question 3

```
1 class Shape:
2     def area(self):
3         raise NotImplementedError()
4
5 class Rectangle(Shape):
6     def __init__(self,w,h):
7         self._w, self._h = w, h
8     def area(self):
9         return self._w * self._h
10
11 def show(s:Shape):
12     print(s.area())
13
14 show(Rectangle(2,3))
15 show(Shape())
```

What happens on the second call in line 15?

- A. Prints 6 again
- B. area() returns None
- C. Raises NotImplementedError
- D. TypeError at runtime

(2 marks)

Question 4

```
class Counter:
    def __init__(self):
        self._c = 0
    @property
    def count(self):
        self._c += 1
        return self._c

c = Counter()
vals = [c.count for _ in range(3)]
print(sum(vals))
```

What is the print output?

- A. 0
- B. 3
- C. 6
- D. AttributeError

(2 marks)

Question 5

```
class Line:
    def __init__(self,p):
        self._points = [p]
    def add(self,p):
        self._points.append(p)
p = [0,0]
L1 = Line(p)
L2 = Line(p)
p[0]=9
print(L1._points[0][0], L2._points[0][0])
```

Which statement is TRUE?

- A. Outputs "9 9" because both lines reference the same list object
- B. Outputs "9 0" because lists are copied in constructor
- C. Outputs "0 0" because assignment to p[0] is after construction
- D. Raises TypeError

(2 marks)

Question 6

```
import tkinter as tk, tkinter.messagebox as mg
class App:
    def __init__(self):
        self._win=tk.Tk()
        self._e=tk.Entry(self._win)
        self._e.pack()
        tk.Button(self._win,text="OK",command=self.validate).pack()
        self._win.mainloop()
    def validate(self):
        try:
            int(self._e.get())
        except ValueError:
            mg.showerror("Bad!","Not integer")
        else:
            print("good")
App()
```

If the user types "42x" and clicks OK, which path executes?

- A. except only
- B. else only
- C. finally then else
- D. try then else

(2 marks)

Question 7

```
try:
    print('A')
    result = 1/0
except ZeroDivisionError:
    print('B')
else:
    print('C')
finally:
    print('D')
```

What is the complete output?

- A. A B
- B. A B C
- C. A B D
- D. A B C D

(2 marks)

Question 8

Polymorphism allows the same ...

- A. attribute name for different classes
- B. method call to execute class-specific behaviour
- C. object to become a class variable
- D. GUI layout manager to be reused

(2 marks)

Question 9

```
class Vec:
    def __init__(self,n):
        self.n = n
    def __add__(self, other):
        if isinstance(other, Vec):
            return Vec(self.n + other.n)
```

```
v = Vec(3) + 4
print(v.n)
```

The last line results in ...

- A. v becomes Vec(7)
- B. TypeError
- C. NotImplemented returned silently
- D. AttributeError

(2 marks)

Question 10

```
class A:
    def __str__(self):
        return f"{self}"
print(A())
```

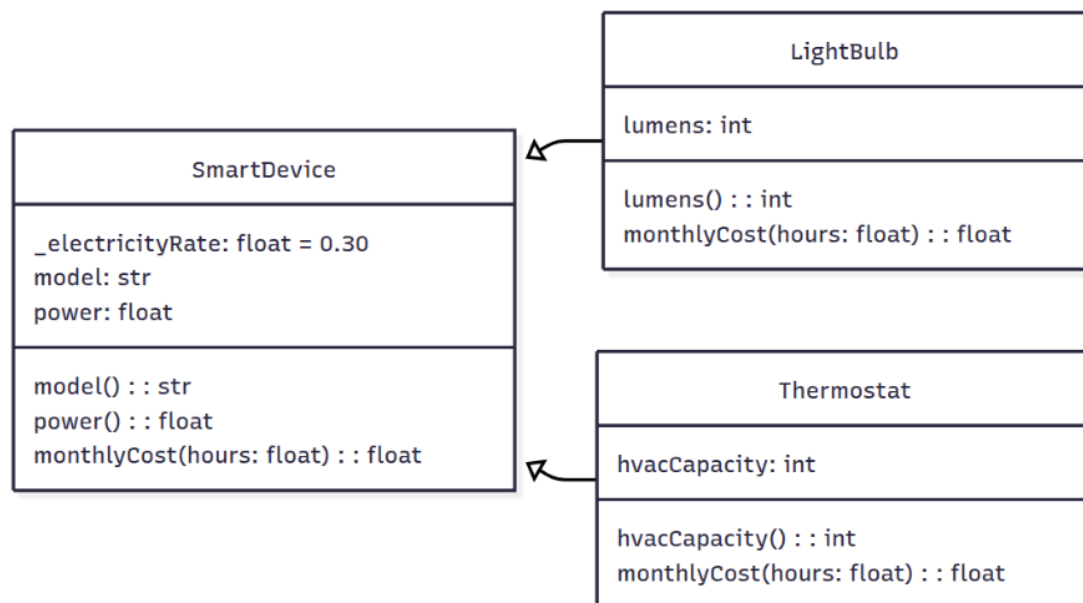
Choosing the BEST outcome:

- A. Prints the object's id string
- B. RecursionError because __str__ continually calls itself
- C. TypeError: f-string not allowed in __str__
- D. Prints the class name

(2 marks)

Question 11

SmartHome Pte Ltd sells different smart devices. Every device has a model-number and power rating (in W). Certain devices (LightBulb, Thermostat) add extra behaviour. Implement the class in the following questions.



Question 11a

Design an abstract superclass SmartDevice with:

- protected attributes model and power
- class variable `_electricityRate = 0.30` (\$/kWh)
- getter properties for both attributes

- an abstract method `monthlyCost(hours)` returning energy-cost for the usage-hours supplied

(5 marks)

Question 11b

Implement subclasses and demonstrate your understanding through the following:

LightBulb:

- extra attribute `_lumens`; same constructor arguments as `SmartDevice` plus lumens.
- Getter property for the new attribute
- Monthly cost is $\text{hours} \times \text{power}/1000 \times \text{electricityRate}$.

Thermostat:

- extra attribute `_hvacCapacity`;
- monthly cost is $\text{hours} \times 1.2 \times \text{power}/1000 \times \text{electricityRate}$.

(10 marks)

Question 11c

Show output produced by the script:

```
lb = LightBulb("LB-900", 9, 800)
th = Thermostat("TH-2200", 60, 12000)

for d in (lb, th):
    print(type(d).__name__, ":", f"${d.monthlyCost(150):.2f}")
```

(5 marks)

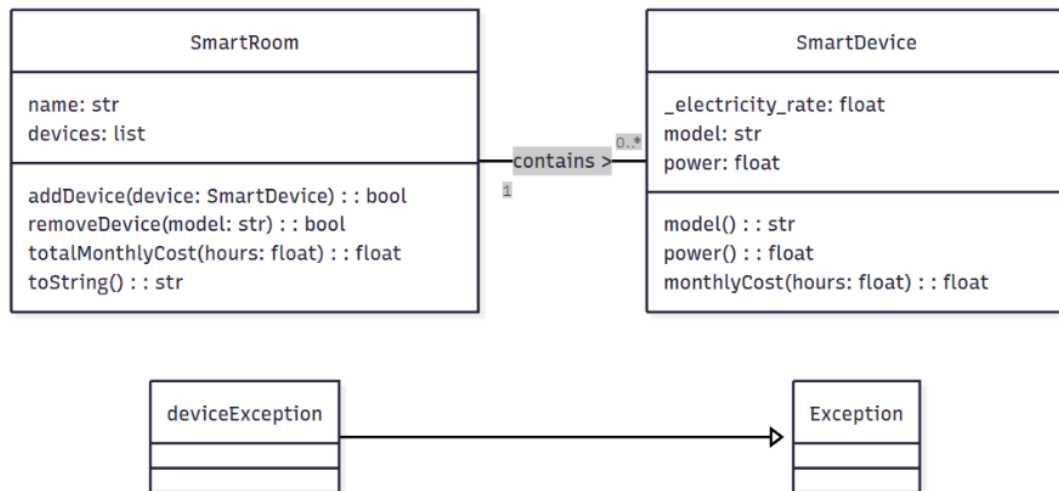
Question 11d

Explain the polymorphism in the loop in part c.

(5 marks)

Question 12

A SmartRoom “has many” SmartDevice objects.



Construct your class with reference to the UML above in the following questions.

Question 12a

Write class SmartRoom with instance variables `_name` (str) and `_devices` (list). Provide:

- `addDevice(device)` – adds a SmartDevice, avoiding duplicates (check by model-number). Raise a `deviceException` when there is a duplicate (same model-number). return True if successful.
- `removeDevice(model)` – removes by model-number, return True/False.
- `totalMonthlyCost(hours)` – sum of each device’s `monthlyCost(hours)`.
- `toString` – nicely list all models in the room, refer to the demo output in part Q12(c).

(16 marks)

Question 12b

Describe **TWO (2)** advantages of using object composition here rather than inheritance.
(4 marks)

Question 12c

Given objects created in part a), fill in the blanks

```
# fill in the blanks

print(bedroom)
print(f"Total monthly cost for '{bedroom._name}' (150 hours): ${total_bedroom_cost:.2f}")

✓ 0.0s
```

```
SmartRoom 'Master Bedroom' contains devices: LB-900, TH-2200.
Total monthly cost for 'Master Bedroom' (150 hours): $3.64
```


Hints: Create the instances necessary for a SmartRoom and name it "bedroom", and add the necessary devices that will produce the output shown above.

(10 marks)

Question 13

Develop a tkinter GUI class SmartRoomGUI that lets a user enter room-name, choose usage-hours and display the total monthly cost computed by a SmartRoom object (assume devices already added).

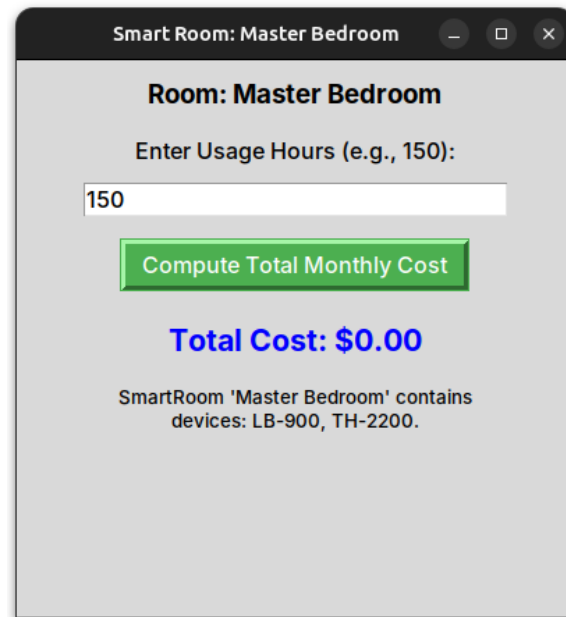


Figure 13.1 Default appearance.

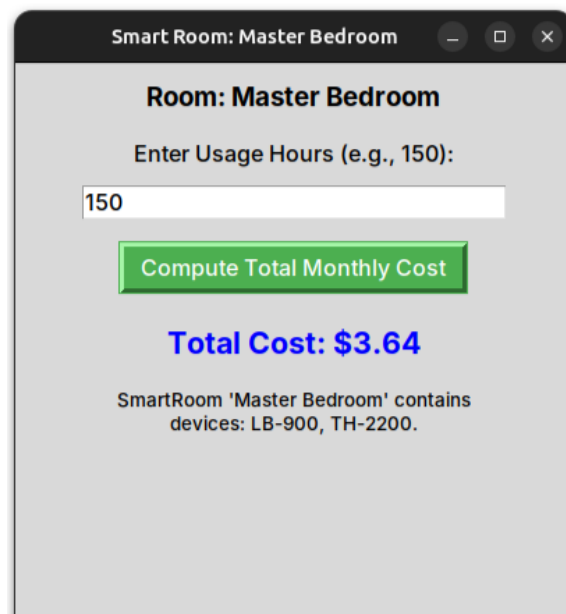


Figure 13.2 After computing the total monthly cost.

Question 13a

```
1 import tkinter as tk
2 from tkinter import messagebox # For showing error messages
3 class SmartRoomGUI:
4     def __init__(self, smart_room: SmartRoom):
5         self._root = tk.Tk()
6         self._root.geometry("400x400")
7         self._room = smart_room
8         # here
9
10        # here
11        self._room_name_label.pack(pady=10)
12
13        self._hours_label = tk.Label(self._root, text="Enter Usage Hours (e.g., 150):", font=("Inter", 12))
14        self._hours_label.pack(pady=5)
15
16        self._hours_entry = tk.Entry(self._root, width=30, font=("Inter", 12))
17        self._hours_entry.pack(pady=5)
18        self._hours_entry.insert(0, "150")
19
20        self._compute_button = tk.Button(
21            self._root, text="Compute Total Monthly Cost", command=self.computeCost,
22            font=("Inter", 12), bg="#4CAF50", fg="white", activebackground="#45a049",
23            relief=tk.RAISED, bd=3, highlightbackground="#4CAF50"
24        )
25        self._compute_button.pack(pady=10)
26
27        self._cost_display_var = tk.StringVar()
28        self._cost_display_var.set("Total Cost: $0.00")
29        # here
30        self._cost_label.pack(pady=10)
31
32        # here
33        # here
34
35        self._root.mainloop()
36
37    def computeCost(self):
38        # complete here
39        pass
```

Please complete the lines 8, 10, 29, 32 and 33 to achieve the interface shown in Figure 13.1.

(10 marks)

Question 13b

Write computeCost and complete line 38 so that it:

- validates that the hours field is a positive number, otherwise pops a messagebox.showerror
- calls room.totalMonthlyCost(hours) and shows result to two decimal places
- demonstrates exception handling where appropriate.

Refer to Figures below:

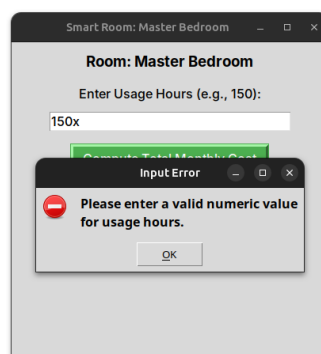


Figure 13.3

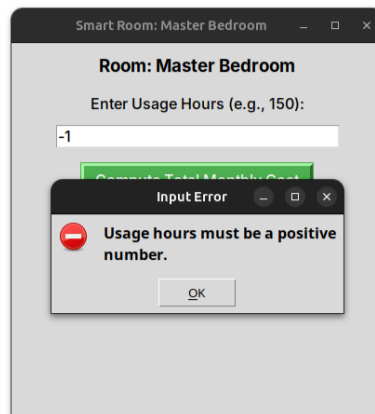


Figure 13.4

Reference

`tkinter.messagebox.showerror(title=None, message=None, **options)`

Creates and displays an error message box with the specified title and message.

(15 marks)

----- END OF PAPER -----